

A. Các Mẫu Tạo (Creational Patterns)

Tập trung vào cách tạo ra đối tượng một cách linh hoạt.

1. Factory Method (Phương thức Nhà máy):

- **Mục đích:** Để lớp con quyết định sẽ tạo ra đối tượng cụ thể nào.
- **Từ khóa:** *Lớp con quyết định, một loại sản phẩm, ghi đè phương thức tạo.*

2. Abstract Factory (Nhà máy Trừu tượng):

- **Mục đích:** Cung cấp một giao diện để tạo ra các **họ (families)** đối tượng liên quan mà không cần chỉ định lớp cụ thể của chúng.
- **Từ khóa:** *Họ sản phẩm, nhiều loại sản phẩm liên quan, siêu nhà máy, nhà máy con.*

3. Builder (Thợ xây):

- **Mục đích:** Tách biệt quá trình xây dựng một đối tượng phức tạp khỏi biểu diễn của nó.
- **Từ khóa:** *Đối tượng phức tạp, xây dựng từng bước, nhiều biểu diễn từ cùng quy trình.*

4. Prototype (Nguyên mẫu):

- **Mục đích:** Tạo đối tượng mới bằng cách sao chép (clone) một đối tượng đã có.
- **Từ khóa:** *Sao chép, nhân bản, đối tượng mẫu, tiết kiệm chi phí tạo mới.*

5. Singleton (Độc bản):

- **Mục đích:** Đảm bảo một lớp chỉ có duy nhất một thể hiện và cung cấp điểm truy cập toàn cục đến nó.
- **Từ khóa:** *Một thể hiện duy nhất, truy cập toàn cục, constructor private.*

B. Các Mẫu Cấu trúc (Structural Patterns)

Tập trung vào cách các lớp và đối tượng được kết hợp lại để tạo thành các cấu trúc lớn hơn.

1. Adapter (Cục chuyển):

- **Mục đích:** Làm cho giao diện của một lớp không tương thích trở nên tương thích với giao diện mà client mong đợi.
- **Từ khóa:** *Bọc, giao diện không tương thích, chuyển đổi giao diện.*

2. Bridge (Cầu nối):

- **Mục đích:** Tách biệt một sự trừu tượng hóa (abstraction) khỏi phần triển khai (implementation) của nó để cả hai có thể thay đổi độc lập.
- **Từ khóa:** *Tách biệt abstraction và implementation, thay đổi độc lập, tránh bùng nổ lớp con.*

3. Composite (Hợp chất):

- **Mục đích:** Tổ chức các đối tượng thành cấu trúc cây để biểu diễn cây bộ phận-tổng thể, cho phép client xử lý đối tượng riêng lẻ và tổ hợp một cách thống nhất.

- **Từ khóa:** *Cấu trúc cây, bộ phận-tổng thể, lá (leaf), nhánh (composite), xử lý đồng nhất.*

4. Decorator (Tô điểm):

- **Mục đích:** Gán thêm trách nhiệm/chức năng cho một đối tượng một cách động tại thời điểm chạy, linh hoạt hơn kế thừa.
- **Từ khóa:** *Bọc, thêm chức năng động, không thay đổi giao diện gốc, xếp chồng.*

5. Facade (Mặt tiền):

- **Mục đích:** Cung cấp một giao diện đơn giản, hợp nhất cho một tập hợp các giao diện phức tạp trong một hệ thống con.
- **Từ khóa:** *Giao diện đơn giản, che giấu phức tạp, hệ thống con.*

6. Flyweight (Trọng lượng bay):

- **Mục đích:** Sử dụng việc chia sẻ để hỗ trợ hiệu quả một số lượng lớn các đối tượng nhỏ có nhiều điểm chung.
- **Từ khóa:** *Chia sẻ đối tượng, tiết kiệm bộ nhớ, trạng thái nội tại (intrinsic), trạng thái ngoại tại (extrinsic).*

7. Proxy (Đại diện):

- **Mục đích:** Cung cấp một đối tượng thay thế hoặc đại diện cho một đối tượng khác để kiểm soát việc truy cập đến nó.
- **Từ khóa:** *Đối tượng thay thế, kiểm soát truy cập, remote, virtual, protection, smart proxy.*

C. Các Mẫu Hành vi (Behavioral Patterns)

Tập trung vào cách các đối tượng tương tác và phân chia trách nhiệm.

1. Chain of Responsibility (Chuỗi Trách Nhiệm):

- **Mục đích:** Tránh việc gắn kết chặt chẽ giữa người gửi yêu cầu và người nhận bằng cách cho nhiều đối tượng cơ hội xử lý yêu cầu, truyền yêu cầu dọc theo chuỗi.
- **Từ khóa:** *Chuỗi xử lý, truyền yêu cầu, không biết người xử lý cuối cùng.*

2. Command (Lệnh):

- **Mục đích:** Đóng gói một yêu cầu thành một đối tượng, cho phép tham số hóa client, xếp hàng, ghi log, và hỗ trợ undo/redo.
- **Từ khóa:** *Đóng gói yêu cầu, đối tượng lệnh, Invoker, Receiver, undo/redo.*

3. Interpreter (Thông Dịch/Diễn Dịch):

- **Mục đích:** Định nghĩa một biểu diễn ngữ pháp cho một ngôn ngữ và một trình thông dịch để diễn giải các câu trong ngôn ngữ đó.
- **Từ khóa:** *Ngữ pháp, ngôn ngữ riêng, cây cú pháp trừu tượng (AST), diễn giải câu lệnh.*

4. Iterator (Biến Duyệt/Bộ Lặp):

- **Mục đích:** Cung cấp một cách để truy cập tuần tự các phần tử của một đối tượng tập hợp mà không để lộ cấu trúc bên trong của nó.
- **Từ khóa:** *Duyệt tập hợp, truy cập tuần tự, không lộ cấu trúc, next(), hasNext().*

5. Mediator (Người Điều Phối/Trung Gian):

- **Mục đích:** Định nghĩa một đối tượng đóng gói cách một tập hợp các đối tượng (Colleagues) tương tác, giảm sự phụ thuộc trực tiếp giữa chúng.
- **Từ khóa:** *Điều phối tương tác, giảm phụ thuộc trực tiếp, Colleagues, giao tiếp qua trung gian.*

6. Memento (Bản Ghi Nhớ/Bản Sao Trạng Thái):

- **Mục đích:** Lưu trữ và khôi phục trạng thái bên trong của một đối tượng mà không vi phạm tính đóng gói.
- **Từ khóa:** *Lưu trạng thái, khôi phục trạng thái, Originator, Caretaker, undo/redo.*

7. Observer (Người Quan Sát):

- **Mục đích:** Định nghĩa một sự phụ thuộc một-nhiều giữa các đối tượng sao cho khi một đối tượng (Subject) thay đổi trạng thái, tất cả các đối tượng phụ thuộc (Observers) của nó sẽ được thông báo và cập nhật tự động.
- **Từ khóa:** *Đăng ký nhận thông báo, Subject, Observer, cập nhật tự động, một-nhiều.*

8. State (Trạng Thái):

- **Mục đích:** Cho phép một đối tượng thay đổi hành vi của nó khi trạng thái bên trong của nó thay đổi, giống như nó đã thay đổi lớp của mình.
- **Từ khóa:** *Thay đổi hành vi theo trạng thái, Context, đối tượng trạng thái riêng.*

9. Strategy (Chiến Lược/Chiến Thuật):

- **Mục đích:** Định nghĩa một họ các thuật toán, đóng gói từng thuật toán, và làm cho chúng có thể thay thế lẫn nhau.
- **Từ khóa:** *Họ thuật toán, đóng gói thuật toán, thay thế thuật toán lúc chạy, Context.*

10. Template Method (Phương Thức Khung):

- **Mục đích:** Định nghĩa bộ khung của một thuật toán trong một thao tác, trì hoãn một số bước cho các lớp con.
- **Từ khóa:** *Khung thuật toán, lớp cha định nghĩa bước chung, lớp con tùy chỉnh bước cụ thể, kế thừa.*

11. Visitor (Khách Thăm):

- **Mục đích:** Biểu diễn một thao tác sẽ được thực hiện trên các phần tử của một cấu trúc đối tượng, cho phép thêm thao tác mới mà không thay đổi các lớp của cấu trúc.
- **Từ khóa:** *Thao tác trên cấu trúc đối tượng, không thay đổi lớp gốc, double dispatch, accept(), visit().*